

Componente de Configuración – parte 2



A continuación, vamos a ver los cambios necesarios para que el código funcione correctamente con la última versión de Angular y Firestore.

Código Actualizado configuracion.component.html:

```
<header id="main-header" class="py-2 bg-info text-white">
  <div class="container">
    <div class="row">
      <div class="col-md-6">
        <h1>
          <i class="fas fa-user"></i> Configuración
        </h1>
      </div>
    </div>
  </div>
</header>

<section id="actions" class="py-4 mb-4 bg-light">
  <div class="container">
    <div class="row">
      <div class="col-md-3">
        <a routerLink="/" class="btn btn-primary w-100">
          <i class="fas fa-arrow-left"></i> Regresar al Inicio
        </a>
      </div>
    </div>
  </div>
</section>
```

```

        </div>
        <div class="col-md-3">
            <button type="submit" (click)="guardar()" class="btn btn-success w-100">
                <i class="fas fa-check"></i> Guardar Cambios
            </button>
        </div>
    </div>
</div>
</section>

<section id="configuracion">
    <div class="container">
        <div class="row">
            <div class="col">
                <div class="card">
                    <div class="card-header">
                        <h4>Editar Configuración</h4>
                    </div>
                    <div class="card-body">
                        <fieldset class="mb-3">
                            <legend>Permitir Registro de Usuarios</legend>
                            <div class="form-check">
                                <input
                                    type="checkbox"
                                    class="form-check-input"
                                    name="permitirRegistro"
                                    [(ngModel)]="permitirRegistro"
                                    [checked]="permitirRegistro">
                                <label class="form-check-label">
                                    {{permitirRegistro ? "Si" : "No"}}
                                </label>
                            </div>
                        </fieldset>
                    </div>
                </div>
            </div>
        </div>
    </div>
</section>

```

Código Actualizado configuracion.component.ts:

```

import { Component, OnInit } from '@angular/core';
import { Router } from '@angular/router';

```

```

import { ConfiguracionServicio } from '../../servicios/configuracion.service';
import { Configuracion } from '../../modelo/configuracion.model';

@Component({
  selector: 'app-configuracion',
  templateUrl: './configuracion.component.html',
  styleUrls: ['./configuracion.component.css']
})
export class ConfiguracionComponent implements OnInit {

  permitirRegistro = false;

  constructor(
    private router: Router,
    private configuracionServicio: ConfiguracionServicio
  ) { }

  ngOnInit() {
    this.configuracionServicio.getConfiguracion().subscribe(
      (configuracion: Configuracion | null) => {
        this.permitirRegistro = configuracion?.permitirRegistro ?? false;
      }
    )
  }

  guardar(){
    let configuracion = {permitirRegistro: this.permitirRegistro};
    this.configuracionServicio.modificarConfiguracion(configuracion);
    this.router.navigate(['/']);
  }
}

```

Código Actualizado configuración.service.ts:

```

import { Configuracion } from '../../modelo/configuracion.model';
import { Observable } from 'rxjs';
import { Injectable } from '@angular/core';
import { AngularFireStoreDocument, AngularFireStore } from '@angular/fire/compat/firestore';
import { map } from 'rxjs/operators';

@Injectable()

```

```

export class ConfiguracionServicio {
  configuracionDoc: AngularFirestoreDocument<Configuracion>;
  configuracion: Observable<Configuracion | null>; // Cambiado a Configuracion | null

  //id unico de la coleccion de configuracion
  id = '1';

  constructor(private firestore: AngularFirestore) {}

  getConfiguracion(): Observable<Configuracion | null> {
    this.configuracionDoc = this.firestore.doc<Configuracion>(`configuracion/${this.id}`);
    this.configuracion = this.configuracionDoc.valueChanges().pipe(
      map(configuracion => configuracion ? configuracion : null) // Manejar el caso de
undefined
    );
    return this.configuracion;
  }

  modificarConfiguracion(configuracion: Configuracion) {
    this.configuracionDoc = this.firestore.doc<Configuracion>(`configuracion/${this.id}`);
    this.configuracionDoc.update(configuracion);
  }
}

```

Aquí tienes un resumen de los cambios que se hicieron y la razón detrás de cada uno:

1. Manejo de undefined en el Servicio (configuracion.service.ts)

Código Original:

```

getConfiguracion(): Observable<Configuracion> {
  this.configuracionDoc =
this.firestore.doc<Configuracion>(`configuracion/${this.id}`);
  this.configuracion = this.configuracionDoc.valueChanges();
  return this.configuracion;
}

```

Código Modificado:

```

getConfiguracion(): Observable<Configuracion | null> {
  this.configuracionDoc =
this.firestore.doc<Configuracion>(`configuracion/${this.id}`);
  this.configuracion = this.configuracionDoc.valueChanges().pipe(
    map(configuracion => configuracion ? configuracion : null)
  );
  return this.configuracion;
}

```

Cambios Realizados:

- **Cambio en el Tipo de Retorno:** Se cambió el tipo de retorno del método `getConfiguracion()` de `Observable<Configuracion>` a `Observable<Configuracion | null>`.
- **Manejo de undefined:** Se agregó un operador `map` para convertir `undefined` en `null` si el documento no existe.

Razón:

- Firestore devuelve `undefined` si el documento no existe, pero tu código original esperaba siempre un `Configuracion`. Esto provocaba un error de tipo en TypeScript. Al manejar el caso de `undefined`, se asegura que el código no falle cuando el documento no está presente.

2. Manejo de undefined en el Componente (configuracion.component.ts)**Código Original:**

```
ngOnInit() {  
  this.configuracionServicio.getConfiguracion().subscribe(  
    (configuracion: Configuracion) => {  
      this.permitirRegistro = configuracion.permitirRegistro;  
    }  
  )  
}
```

Código Modificado:

```
ngOnInit() {  
  this.configuracionServicio.getConfiguracion().subscribe(  
    (configuracion: Configuracion | null) => {  
      this.permitirRegistro = configuracion?.permitirRegistro ?? false;  
    }  
  );  
}
```

Cambios Realizados:

- **Uso de Tipos Opcionales:** Se cambió el tipo de `configuracion` en la suscripción para que acepte `Configuracion | null`.
- **Operador de Coalescencia Nula (??):** Se usó `configuracion?.permitirRegistro ?? false` para manejar el caso donde `permitirRegistro` podría ser `undefined`.

Razón:

- `permitirRegistro` podría ser `undefined` si `configuracion` es `null` o si no tiene el campo `permitirRegistro`. El uso de la coalescencia nula asegura que `permitirRegistro` siempre sea un valor booleano (`false` si es `undefined`).

Resumen

- **Manejo de Valores `undefined` y `null`:** Ambos cambios se enfocaron en asegurar que el código maneje correctamente los valores que podrían ser `undefined` o `null`, lo cual es común cuando trabajas con datos provenientes de Firestore.
- **Tipado Más Seguro:** Los tipos fueron ajustados para reflejar las posibles realidades de los datos, mejorando la seguridad y estabilidad del código.
- **Uso de Operadores Modernos:** El operador de coalescencia nula (`??`) fue utilizado para garantizar que los valores tengan un valor predeterminado en caso de ser `undefined`.

Estos cambios ayudan a prevenir errores y aseguran que la aplicación maneje adecuadamente los datos provenientes de Firestore, incluso en casos donde los datos pueden estar incompletos o ausentes.

Por último, no es necesario agregar nada más al archivo `app.module.ts`, ya no se requiere en la última versión de Angular.

Recuerden que pueden descargar el código actualizado en la sección de recursos de esta lección.

Saludos!

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](http://www.globalmentoring.com.mx)